## Nombre: Angel Camilo Pachón Padilla

## Resumen e Introducción

En este taller implementamos en Python (sobre Google Colab) cinco algoritmos de ordenamiento: inserción, burbuja, selección, mezcla y quicksort. Después medimos cuánto tarda cada uno en ordenar muestras de distintos tamaños, tomadas de un vector de 500.000 números enteros. El objetivo es evaluar de forma empírica la complejidad temporal de cada algoritmo y compararla con la que predice la teoría: $\mathcal{O}(n^2)$ para inserción, burbuja y selección, y $\mathcal{O}(n \log n)$ para mezcla y quicksort. Para eso promediamos cinco ejecuciones por cada combinación algoritmo–muestra, graficamos los tiempos y ajustamos una recta en escala log-log para estimar el exponente de crecimiento de cada curva. Los exponentes que salieron (cerca de 2 para los tres primeros y cerca de 1 para los dos últimos) concuerdan con lo esperado.

## Marco Teórico

Todas las funciones de `ordenamientos.py` reciben una lista y devuelven una copia ordenada (empiezan con `arr.copy()`), así que la lista original no se modifica. Esa copia cuesta $\mathcal{O}(n)$ y no cambia el orden de crecimiento de ningún algoritmo. En lo que sigue, $n$ es el número de elementos de la lista.

### Ordenamiento por inserción (*Insertion Sort*, función `insercion`)

Es parecido a ordenar cartas en la mano: se toma el elemento de la posición $i$ y se inserta en su lugar dentro de la parte que ya está ordenada, corriendo hacia la derecha todos los que sean mayores (ese es el `while` del código). Para cada $i$, el `while` puede mover hasta $i$ elementos. En el peor caso (lista en orden inverso) se hacen

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

desplazamientos. Con datos aleatorios, en promedio se mueve más o menos la mitad de la parte ordenada, o sea unos $n^2/4$ desplazamientos en total, que sigue siendo $\mathcal{O}(n^2)$. Solo en el mejor caso (lista ya ordenada) baja a $\mathcal{O}(n)$.

### Ordenamiento burbuja (*Bubble Sort*, función `burbuja`)

Compara vecinos y los intercambia si están en mal orden; en cada pasada el elemento más grande "flota" hasta el final. El ciclo externo da $n-1$ vueltas y el interno recorre $n-1-i$ posiciones, así que el número de comparaciones es

$$\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}.$$

Como el código no tiene ninguna bandera para parar cuando la lista ya quedó ordenada, hace siempre esas comparaciones, y por eso es $\mathcal{O}(n^2)$ incluso en el mejor caso. Además, con datos aleatorios hace muchos intercambios (del orden de $n^2/4$), lo cual lo deja con una constante más grande que los otros dos cuadráticos.

### Ordenamiento por selección (*Selection Sort*, función `seleccion`)

En cada vuelta busca el mínimo de la parte desordenada (`for j in range(i + 1, n)`) y lo intercambia con la posición $i$. La búsqueda del mínimo hace exactamente las mismas $n(n-1)/2$ comparaciones que la burbuja, sin importar cómo venga la lista, así que es $\mathcal{O}(n^2)$ en todos los casos. La diferencia es que hace como máximo $n-1$ intercambios, muchos menos que la burbuja.

### Ordenamiento por mezcla (*Merge Sort*, función `mezcla`)

Usa la estrategia de divide y vencerás: parte la lista en dos mitades (`arr[:medio]` y `arr[medio:]`), ordena cada mitad con una llamada recursiva y luego junta las dos mitades ya ordenadas con la función auxiliar `_mezclar`, que recorre cada elemento una sola vez, es decir, $\Theta(n)$. El tiempo cumple la recurrencia

$$T(n) = 2\,T\Big(\frac{n}{2}\Big) + \Theta(n).$$

Hay unos $\log_2 n$ niveles de recursión y en cada nivel se hace $\Theta(n)$ de trabajo en total, de modo que $T(n) = \Theta(n \log n)$ en el mejor, el promedio y el peor caso. El precio es memoria extra $\mathcal{O}(n)$, porque crea listas nuevas todo el tiempo.

### Quicksort (función `quicksort`)

También es divide y vencerás. Toma el primer elemento como pivote, separa los demás en `menores` (los $\leq$ pivote) y `mayores` con dos listas por comprensión, y ordena cada lado de forma recursiva. Cada nivel de la recursión recorre los elementos una vez, $\Theta(n)$. Si el pivote deja las dos partes más o menos parejas, la recurrencia es la misma de mezcla, $T(n) = 2\,T(n/2) + \Theta(n)$, y da $\mathcal{O}(n \log n)$; con datos aleatorios este es el caso esperado. El peor caso ocurre cuando el pivote es siempre el mínimo o el máximo (por ejemplo, una lista ya ordenada, porque el pivote es el primer elemento): ahí $T(n) = T(n-1) + \Theta(n)$ y se vuelve $\mathcal{O}(n^2)$. Con datos aleatorios eso es muy poco probable. Esta versión no ordena "en el sitio", porque crea listas nuevas.

## Metodología

El experimento se hizo en un notebook de Google Colab con los siguientes pasos:

1. **Datos.** Se generó un archivo de texto plano (`numeros.txt`) con 500.000 números enteros aleatorios y se cargó completo en memoria.

2. **Verificación.** Antes de medir, el script comprueba con una lista de control de 1.000 datos que los cinco algoritmos ordenan bien y que no modifican la lista de entrada.

3. **Muestras.** Se tomaron muestras de tamaño $n$ del vector. Para los algoritmos $\mathcal{O}(n \log n)$ (mezcla y quicksort) se usaron $n = 500;\ 1.000;\ 10.000;\ 30.000;\ 50.000;\ 100.000;\ 200.000;\ 300.000;\ 400.000$ y 500.000. Para los algoritmos $\mathcal{O}(n^2)$ se usaron $n = 500;\ 1.000;\ 5.000;\ 10.000;\ 20.000$ y 30.000. Ahí agregamos dos tamaños que no estaban en el enunciado (5.000 y 20.000) para tener más puntos en el rango que sí se puede medir.

4. **Medición.** El tiempo de cada ejecución se midió con `time.perf_counter()`. Cada combinación algoritmo–muestra se ejecutó 5 veces y se guardó el promedio, en segundos, en el archivo `resultados.csv`. En total son 38 combinaciones ($2 \times 10$ de los algoritmos linealítmicos más $3 \times 6$ de los cuadráticos).

5. **Por qué los cuadráticos solo llegan a $n = 30.000$.** Por eficiencia computacional: con los tiempos que medimos, una extrapolación cuadrática desde $n = 30.000$ estima unas 3,8 horas para una sola ejecución de burbuja con $n = 500.000$. Sumando las cinco repeticiones de los tres algoritmos cuadráticos serían más de 35 horas de cómputo (es una estimación, no una medición). Además, no habría aportado información nueva: con seis puntos entre $n = 500$ y $n = 30.000$ (un rango de 60 veces) la pendiente de la curva ya queda muy bien definida. Los algoritmos $\mathcal{O}(n \log n)$, en cambio, sí se evaluaron en todo el rango hasta $n = 500.000$.

Los tiempos absolutos dependen de la máquina de Colab y del intérprete de Python; lo que nos interesa es la forma en que crecen las curvas.

# Resultados

## Tiempos promedio

Las tablas de abajo traen los promedios de 5 ejecuciones, en segundos, tal como quedaron en `resultados.csv`. Las separamos por clase de algoritmo porque no se midieron los mismos tamaños (ver Metodología).

Cuadro 1: Tiempo promedio (s) de los algoritmos $\mathcal{O}(n^2)$.

| $n$ | Inserción | Burbuja | Selección |
|---:|---:|---:|---:|
| 500 | 0,009000 | 0,011199 | 0,005274 |
| 1.000 | 0,031912 | 0,045900 | 0,021677 |
| 5.000 | 0,633105 | 1,296445 | 0,540843 |
| 10.000 | 2,636721 | 5,082828 | 2,286190 |
| 20.000 | 11,018668 | 20,936984 | 9,335707 |
| 30.000 | 25,418920 | 49,182165 | 21,441462 |

Cuadro 2: Tiempo promedio (s) de los algoritmos $\mathcal{O}(n \log n)$.

| $n$ | Mezcla | Quicksort |
|---:|---:|---:|
| 500 | 0,000762 | 0,002007 |
| 1.000 | 0,001635 | 0,001292 |
| 10.000 | 0,022931 | 0,014177 |
| 30.000 | 0,099090 | 0,049171 |
| 50.000 | 0,193639 | 0,087911 |
| 100.000 | 0,293746 | 0,194014 |
| 200.000 | 0,634592 | 0,424980 |
| 300.000 | 0,998476 | 0,713879 |
| 400.000 | 1,445753 | 1,018815 |
| 500.000 | 1,820686 | 1,341287 |

Algunas cosas que se ven directo en las tablas:

- En la Tabla 1, con $n = 30.000$ el orden de más lento a más rápido es burbuja (49,18 s), inserción (25,42 s) y selección (21,44 s). Los tres son $\mathcal{O}(n^2)$, pero la burbuja tarda cerca de 1,9

veces lo que tarda inserción y 2,3 veces lo que tarda selección; eso es lo que esperábamos por la cantidad de intercambios que hace.

- Al triplicar el tamaño de 10.000 a 30.000, la burbuja pasó de 5,08 s a 49,18 s, o sea unas 9,7 veces más. Si el crecimiento es cuadrático, triplicar $n$ debería multiplicar el tiempo por $3^2 = 9$; casi exacto.
- En la Tabla 2, mezcla y quicksort ordenan 500.000 números en 1,82 s y 1,34 s. Inserción necesitó 25,42 s para ordenar solo 30.000.
- Al pasar de $n = 100.000$ a $n = 500.000$ (cinco veces más datos), el tiempo de quicksort se multiplicó por 6,9 y el de mezcla por 6,2. Un algoritmo cuadrático habría tardado unas 25 veces más.
- Con tamaños muy pequeños ($n = 500$) los tiempos son de milisegundos: ahí quicksort salió más lento que mezcla, e incluso más lento que él mismo con $n = 1.000$.

### Ajuste de curvas

Para encontrar qué función describe mejor a cada algoritmo se hace una regresión lineal en escala log-log. Si el tiempo crece como $T(n) \approx c \cdot n^p$, al sacar logaritmos queda

$$\log(T) = p \cdot \log(n) + k,$$

que es una recta cuya pendiente es justamente el exponente $p$. Entonces, si $p \approx 2$ el algoritmo es cuadrático, y si $p$ está un poco por encima de 1 el crecimiento es de tipo $n \log n$. Recalculamos este ajuste de mínimos cuadrados con los datos de `resultados.csv` (es el mismo de la última celda del notebook) y los valores coinciden con los de la Tabla 3.

Cuadro 3: Exponente $p$ estimado por regresión log-log para cada algoritmo.

| Algoritmo | Función | $p$ estimado | Puntos usados |
|---|---|---|---|
| Inserción (*Insertion Sort*) | `insercion` | 1,94 | 6 |
| Burbuja (*Bubble Sort*) | `burbuja` | 2,05 | 6 |
| Selección (*Selection Sort*) | `seleccion` | 2,03 | 6 |
| Mezcla (*Merge Sort*) | `mezcla` | 1,13 | 10 |
| Quicksort | `quicksort` | 1,01 | 10 |

Los tres algoritmos cuadráticos quedaron con $p$ entre 1,94 y 2,05, muy cerca de 2. Los otros dos quedaron con $p$ cerca de 1. Aquí hay un detalle: una función $n \log n$ no tiene pendiente exactamente 1 en escala log-log, sino que su pendiente local es $1 + 1/\ln n$, que para estos tamaños da más o menos 1,1. Mezcla (1,13) cuadra bien con eso. En quicksort (1,01) creemos que el punto de $n = 500$, que salió más lento que el de $n = 1.000$, aplana la recta y hace bajar el exponente.

### Lo que muestran las gráficas

La Figura 1 pone los cinco algoritmos juntos en escala log-log. Se ven dos grupos bien separados, y dentro de cada grupo las curvas son casi rectas paralelas: las de inserción, burbuja y selección tienen pendiente cercana a 2 y las de mezcla y quicksort pendiente cercana a 1. La distancia entre

los dos grupos es enorme: con $n = 30.000$ la burbuja (49,18 s) es unas 500 veces más lenta que mezcla (0,099 s) y unas 1.000 veces más lenta que quicksort (0,049 s), o sea, tres órdenes de magnitud.

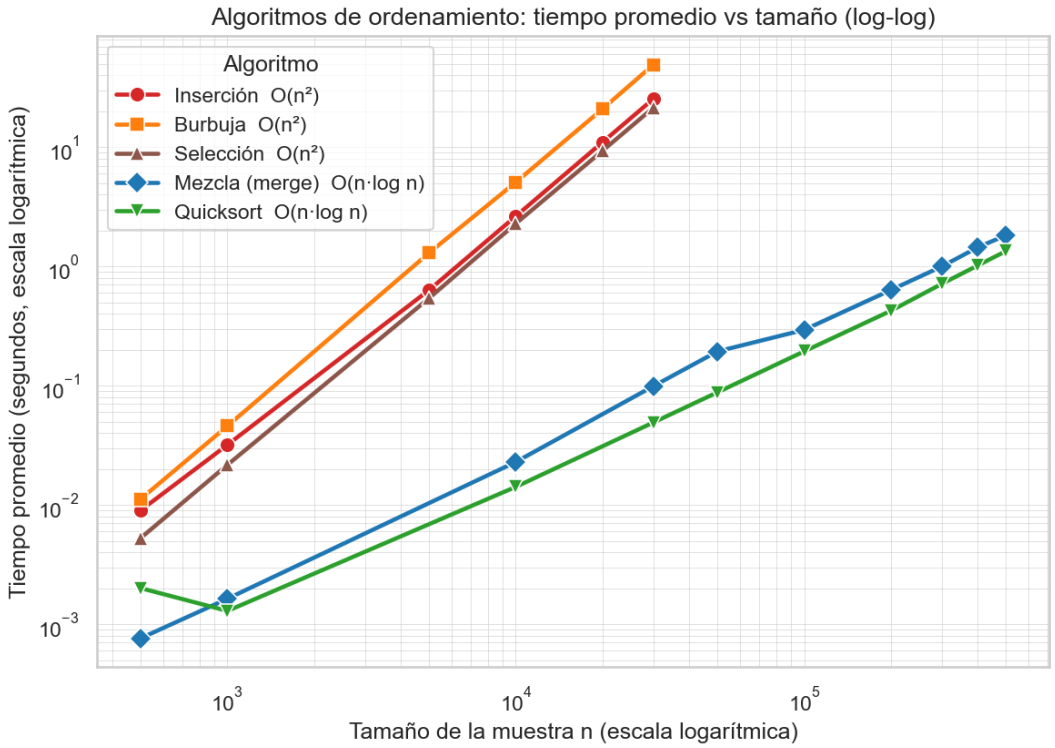



Figura 1: Tiempo promedio vs. tamaño de la muestra para los cinco algoritmos, en escala log-log.

La Figura 2 muestra solo los algoritmos cuadráticos en escala lineal. Las tres curvas se doblan hacia arriba como parábolas, y esa curvatura es la firma de $n^2$. La burbuja queda por encima de todas, inserción en el medio y selección por debajo; hay una diferencia de constantes, pero la forma es la misma.

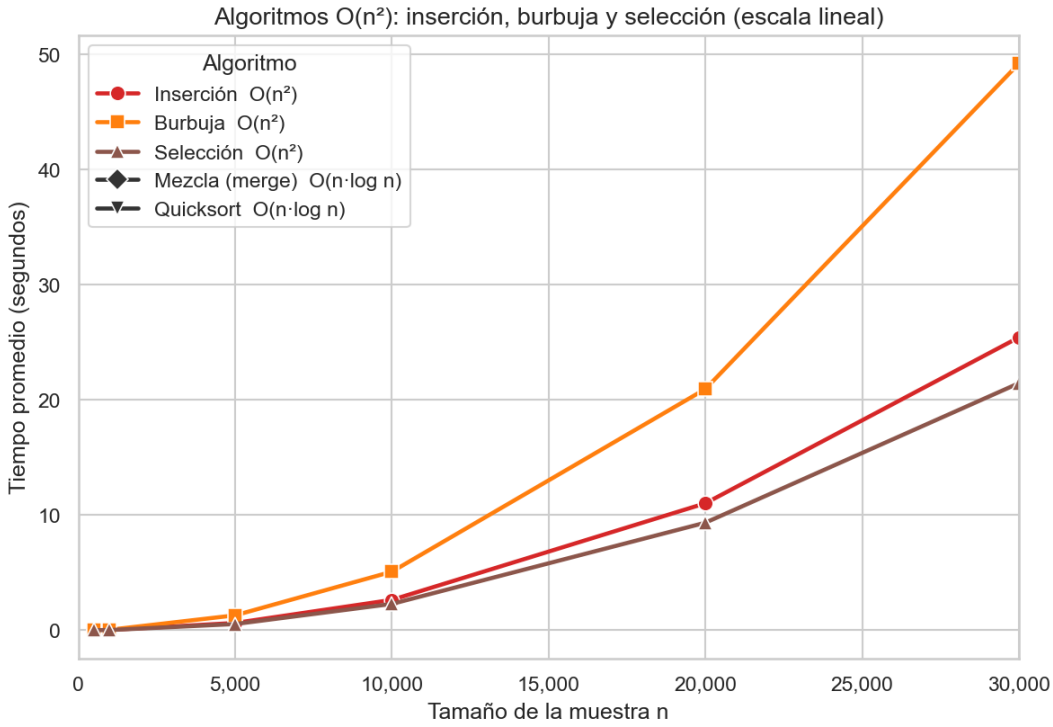



Figura 2: Algoritmos $\mathcal{O}(n^2)$ (inserción, burbuja y selección) en escala lineal, hasta $n = 30.000$.

La Figura 3 muestra mezcla y quicksort en escala lineal hasta $n = 500.000$. Aquí las curvas se ven casi como rectas, porque el factor $\log n$ crece muy despacio. Quicksort va por debajo de mezcla en todos los tamaños grandes ($n \geq 10.000$), con una ventaja de entre 1,4 y 2,2 veces, y la brecha absoluta se ensancha con $n$ (para $n = 500.000$ son unos 0,48 s). Como ambos tienen la misma complejidad, la diferencia viene de las constantes multiplicativas: lo más probable es que mezcla pierda tiempo creando listas nuevas con `arr[:medio]`, `append` y `extend` en cada nivel de la recursión, mientras que quicksort usa listas por comprensión, que en Python son más rápidas. Esto aplica para datos aleatorios, que fueron los que probamos.

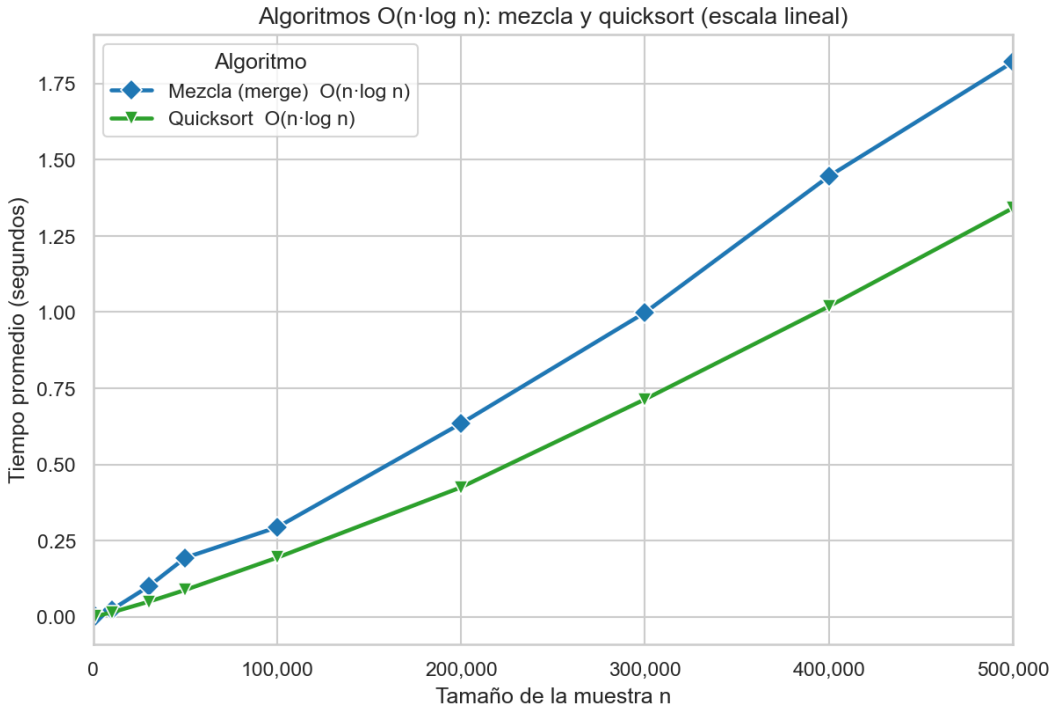



Figura 3: Algoritmos $\mathcal{O}(n \log n)$ (mezcla y quicksort) en escala lineal, hasta $n = 500.000$.

## Conclusiones

1. **¿Qué función se ajusta mejor a cada algoritmo?** Inserción, burbuja y selección se ajustan a una función cuadrática, $T(n) \approx c\,n^2$: sus exponentes empíricos fueron 1,94, 2,05 y 2,03. Mezcla y quicksort se ajustan a $T(n) \approx c\,n \log n$: sus exponentes fueron 1,13 y 1,01, cerca de 1 y muy lejos de 2. Así, los datos validan las complejidades teóricas $\mathcal{O}(n^2)$ y $\mathcal{O}(n \log n)$ que habíamos deducido del código. Una aclaración honesta: el valor de $p$ por sí solo no distingue del todo entre $n$ y $n \log n$; lo que nos hace quedarnos con $n \log n$ es la recurrencia del código y el hecho de que al multiplicar $n$ por 5 el tiempo se multiplicó por más de 6 (y no por 5). Tampoco probamos un ajuste directo de $c\,n \log n$ a los datos; la conclusión se apoya en la pendiente log-log y en la teoría.

2. **¿Qué información nos da la gráfica?** En escala log-log, una función $c\,n^p$ se vuelve una recta cuyo valor de la pendiente es el exponente $p$; por eso se puede leer la complejidad directamente de la inclinación de cada curva. Si dos curvas son paralelas, tienen el mismo orden de crecimiento y solo difieren en la constante $c$ (el desplazamiento vertical): burbuja vs. selección, o quicksort vs. mezcla. En cambio, cuando cambia la pendiente, la diferencia crece con $n$: entre los dos grupos hay un factor de 500 a 1.000 con $n = 30.000$ y seguiría aumentando. La constante de la burbuja es solo unas dos veces peor que la de selección, mientras que pasar de $n^2$ a $n \log n$ cambia por completo lo que se puede ordenar en un tiempo razonable (mezcla ordenó 500.000 datos en menos tiempo que inserción tardó con 30.000). O sea que, para $n$ grande, la complejidad asintótica manda sobre las optimizaciones de bajo nivel.

3. **Limitaciones.** Los tiempos absolutos dependen del equipo y de Python; todas las pruebas se hicieron con datos aleatorios (con listas ya ordenadas, el quicksort con pivote en el primer

elemento se degradaría a $\mathcal{O}(n^2)$ y no lo probamos); y con tamaños muy chicos como $n = 500$ el ruido se nota. Aun así, la tendencia con $n$ grande es clara y coincide con la teoría.